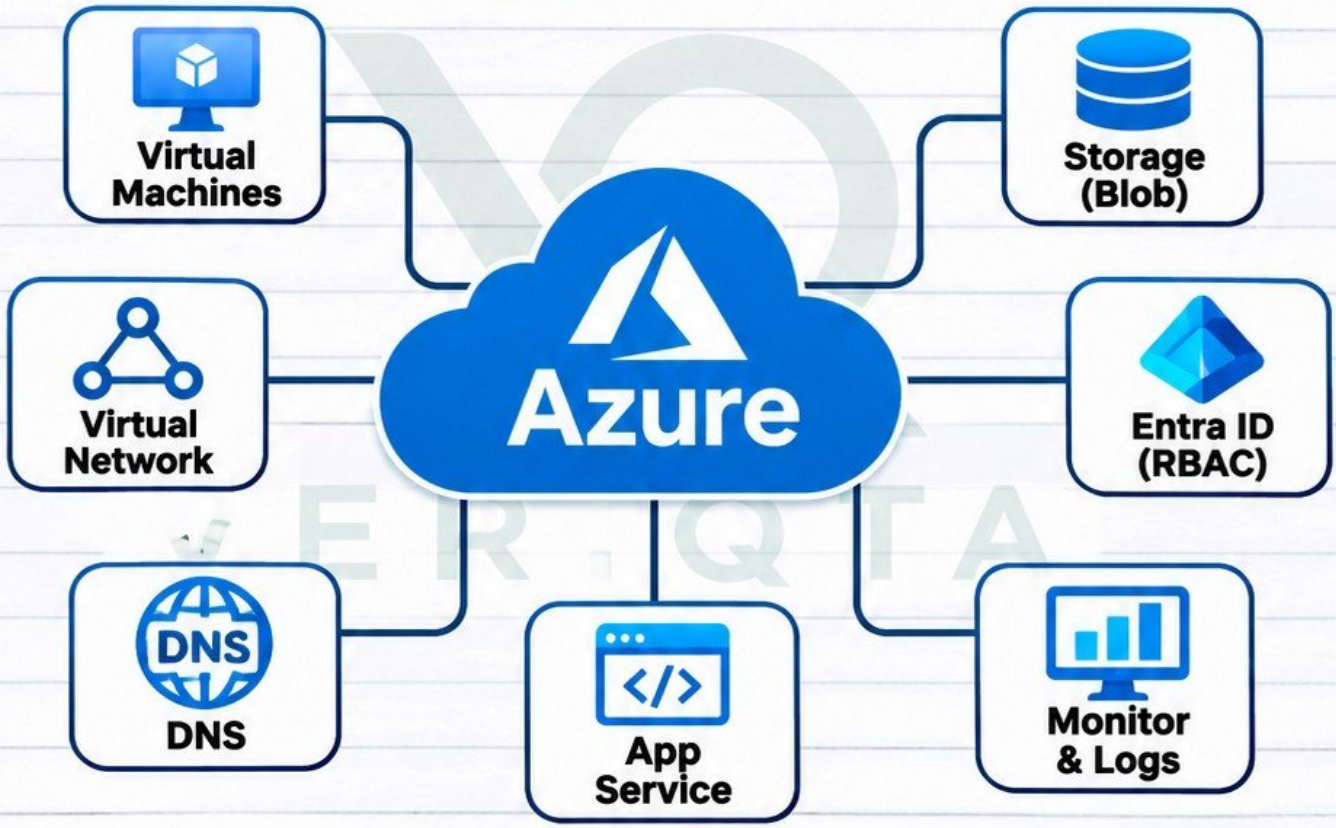
VERIQTA

Azure Cloud for Beginners:

Your First 20 Hands-On Labs



- Build real infrastructure on Microsoft Azure
- Deploy, secure, monitor, and troubleshoot resources
- 20 practical, step-by-step hands-on labs

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Your First Azure Environment: Understand Before You Deploy

- Understand Azure subscriptions, tenants, resource groups, regions, and resources.
- Learn how Azure organizes cloud infrastructure and why billing boundaries matter.

Key Concepts

Tenant:	Your organization identity in Microsoft Entra ID.
Subscription:	A billing and management boundary for your resources.
Resource Group:	A logical container to organize related resources.
Region:	A physical location where Azure runs your resources.
Resource:	An Azure service, for example a VM, storage, or database.



Lab 01: Explore and Create

1. Sign in to the Azure Portal (https://portal.azure.com).
2. Inspect your active subscription (Subscriptions).
3. Create a resource group (Resource groups → Create).
4. Choose a region (e.g., West Europe).
5. Verify the resource group and its location.

Production Lesson

Choosing the wrong subscription can create security, billing, and operational problems.

Azure Cloud Shell: Your First Infrastructure Commands

- Understand Azure Cloud Shell, Azure CLI, Bash, and command-line authentication.
- Learn why engineers use commands instead of relying entirely on the Portal.

Key Concepts

Azure Cloud Shell:	A browser-based terminal in Azure with Azure CLI and Bash preinstalled.
Azure CLI:	A command-line tool to create, manage, and automate Azure resources.
Bash:	A popular Linux shell used to run commands and scripts.
Authentication:	Verifies your identity and permissions to access Azure resources.



Lab 02: Work with Azure Cloud Shell

1. Open Azure Cloud Shell in the Azure Portal (top menu > Cloud Shell).
2. Run: **az account show** (verify your active subscription).
3. Run: **az group list** (list existing resource groups).
4. Run: **az group create --name rg-lab02 \ --location westeurope** (create a new resource group).

Troubleshooting

- **Wrong subscription?**
  Check: **az account show** and switch if needed:
  az account set --subscription <ID>
- **Permission error (AuthorizationFailed)?**
  You may not have the right role. Ask an admin to assign a suitable role (e.g., Contributor) on the subscription or resource group.

Production Insight: Always confirm your active subscription and permissions before creating or changing resources.

Azure Virtual Machines: Deploy Your First Linux Server

- Understand virtual machines, images, VM sizes, operating system disks, and public IP addresses.
- Learn the difference between a virtual machine and a physical server.

Key Concepts

Virtual Machine (VM):	A software-based computer that runs in Azure.
Image:	A prebuilt operating system (e.g., Ubuntu, Windows) used to create a VM.
VM Size:	Defines the CPU, memory (RAM), and performance (e.g., B1s, B2s, D2s).
OS Disk:	The storage that holds the operating system and system files.
Public IP:	A routable IP address used to access your VM from the internet (e.g., SSH).

Physical Server

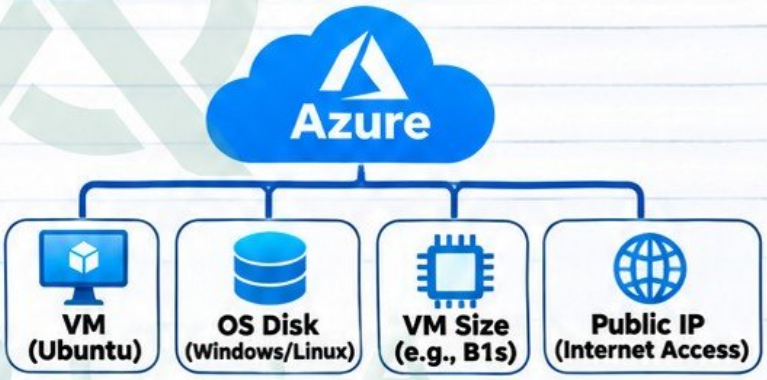


- A real, dedicated machine.
- Fixed capacity (CPU, RAM, Disk).
- Higher cost and slower to provision.

Virtual Machine (VM)



- A virtual (software) computer in Azure.
- Flexible and scalable (e.g., change size).
- Fast to deploy and cost-efficient.



Lab 03: Deploy and Inspect a Linux VM

1. In the Azure Portal, go to Virtual Machines → Create.
2. Choose Ubuntu Server (e.g., 22.04 LTS).
3. Select a VM size (e.g., B1s for practice).
4. Use SSH key authentication (create or upload your public key).
5. Create the VM and wait for deployment.
6. Inspect the VM: connect via SSH and run system checks.

Inspect the VM (Example)

```
$ ssh azureuser@<public-ip>
$ uname -a          # OS details
$ free -h           # Memory usage
$ df -h             # Disk space
$ systemctl status ssh
  # Check SSH service
```

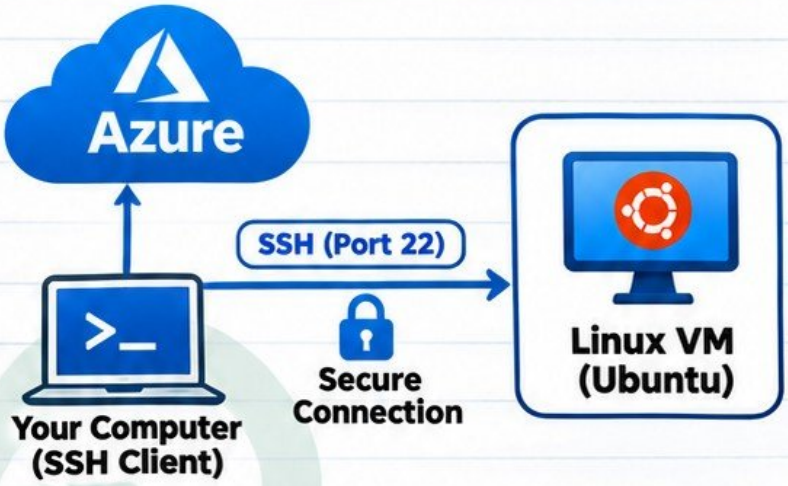
⚠ Production Lesson: Choose appropriately sized resources and avoid unnecessary public exposure.

SSH Into Azure: Connect to and Inspect Your Server

Understand SSH, private keys, usernames, port 22, and secure remote access.

Key Concepts

SSH:	A secure protocol to connect to a remote server over a network.
Private Key:	A secret file used to prove your identity (e.g., id_rsa).
Username:	The login account on the server (e.g., azureuser).
Port 22:	The default port for SSH connections.



Lab 04: Connect and Inspect

1. Connect to the VM using SSH: `ssh azureuser@<public-ip>`
2. Check OS details: `cat /etc/os-release`
3. Check memory usage: `free -h`
4. Check disk space: `df -h`
5. Review running processes: `ps aux --sort=-%mem | head`

SSH Session (Example)

```
$ ssh azureuser@20.121.45.10
azureuser@vm:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="22.04 LTS"
azureuser@vm:~$ free -h
Mem:        3.8G       1.1G       2.7G
azureuser@vm:~$ df -h
Filesystem      Size  Used  Avail
/dev/sda1        29G  6.2G    21G
```

Troubleshooting

Issue	Likely Cause & Fix
Permission denied	• Wrong username or key. • Use the correct user and key: `ssh -i ~/.ssh/id_rsa azureuser@<ip>`.
Connection timed out	• Port 22 blocked by a Network Security Group (NSG) or firewall. • Allow inbound SSH (port 22) to the VM.
Bad key permissions	• Private key too open. • Fix with: `chmod 600 ~/.ssh/id_rsa`.

Security Lesson

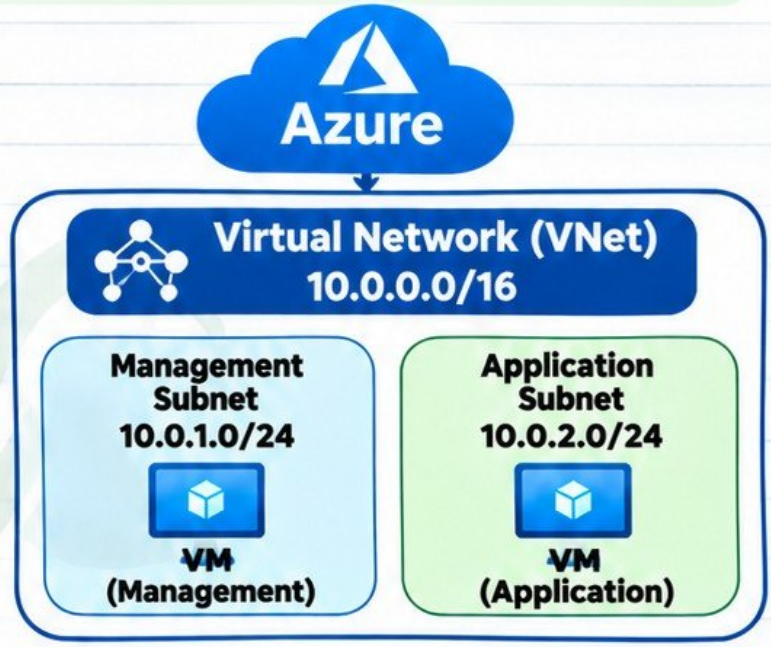
- **Protect your private keys (keep them private and never share them).**
- **Restrict SSH access to trusted networks (e.g., your office or home IP) using an NSG.**

Azure Virtual Networks: Build Your First Private Network

- Understand virtual networks, subnets, private IP addresses, and CIDR notation.
- Learn how Azure resources communicate inside a virtual network.

Key Concepts

Virtual Network (VNet):	A private, isolated network in Azure for your resources.
Subnet:	A smaller network inside a VNet (e.g., app, mgmt).
Private IP:	An internal IP used within your VNet (e.g., 10.0.1.4).
CIDR Notation:	A format that defines an IP range (e.g., 10.0.0.0/16).



Lab 05: Create a Virtual Network

1. In the Azure Portal, go to Virtual Networks → Create.
2. Set the VNet name: vnet-lab.
3. Use address space: 10.0.0.0/16.
4. Add two subnets:
   - mgmt-subnet: 10.0.1.0/24
   - app-subnet: 10.0.2.0/24
5. Create and verify the VNet and subnets in the Overview page.

Production Connection

Network separation (e.g., management and application subnets) helps you:

- Improve security.
- Control access between resources.
- Support future growth (e.g., more apps, databases).

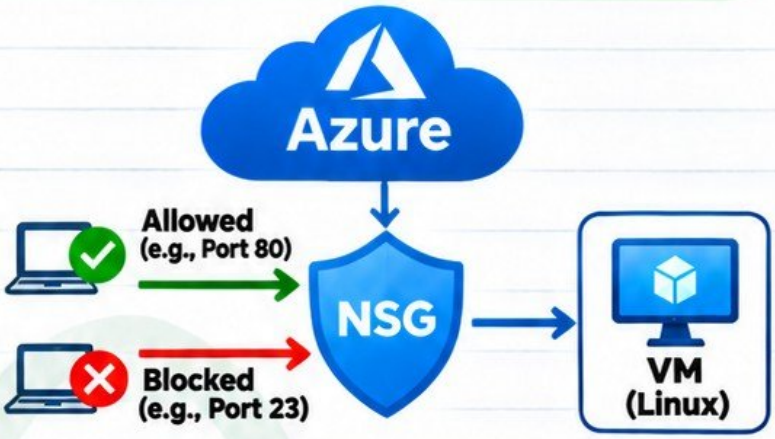
Key Takeaway: A well-designed virtual network gives you a secure, organized, and scalable foundation in Azure.

Network Security Groups: Control Traffic to Your VM

Understand inbound rules, outbound rules, source addresses, destination ports, and rule priorities.

Key Concepts

NSG:	A security layer that controls inbound and outbound traffic to Azure resources (e.g., VMs).
Inbound Rule:	Allows traffic into a resource (e.g., SSH, HTTP).
Outbound Rule:	Allows traffic out from a resource (e.g., to the internet).
Source Address:	The IP or range allowed to connect (e.g., your office IP).
Destination Port:	The port of the service (e.g., 22 for SSH, 80 for HTTP).
Rule Priority:	A number (lower = higher precedence) that decides which rule is applied first.



Example NSG Rules (Inbound)

Priority	Action	Port	Source	Purpose
100	Allow	22	203.0.113.10	SSH (My IP)
200	Allow	80	Any	HTTP (Web)
300	Deny	Any	Any	Block All Other Traffic

Lab 06: Configure an NSG

1. In the Azure Portal, go to Network Security Groups → Create.
2. Name it: nsg-vm-lab and select your resource group and region.
3. Add inbound rules:
   - Allow SSH (Port 22) from your IP (e.g., 203.0.113.10/32).
   - Allow HTTP (Port 80) from Any.
   - Keep the default Deny All rule (priority 65500).
4. Associate the NSG with your VM (Networking → Network security group).
5. Test: access SSH and the web page (http://<public-ip>).

Troubleshooting

Issue	How to Investigate and Fix
Connection blocked	• Check NSG rules and port. • Use Effective security rules in the Azure Portal.
Can't SSH to VM	• Verify your source IP in the rule. • Ensure port 22 is allowed.
Rule not working	• Check rule priority (lower number wins). • Confirm the NSG is associated with the correct VM.

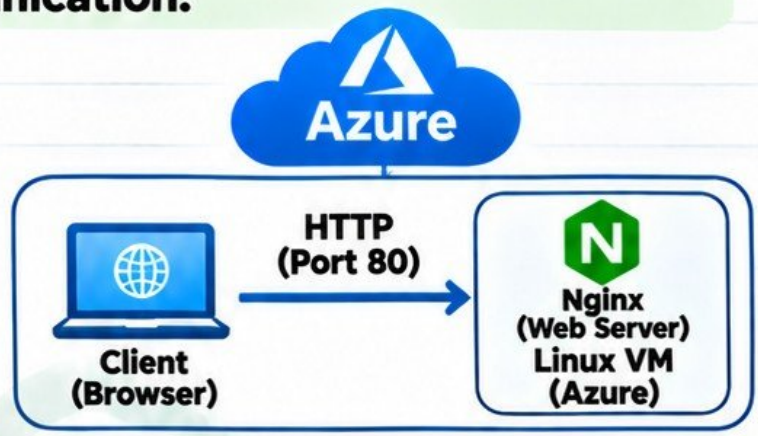
Security Lesson: Never expose every port to the internet for convenience. Allow only the ports and sources you actually need.

Install Nginx: Host Your First Website on Azure

Understand web servers, HTTP, TCP port 80, and client-server communication.

Key Concepts

Web Server:	Software that serves web content to users (e.g., Nginx, Apache).
HTTP:	The protocol used by browsers to request web pages.
TCP Port 80:	The default port for HTTP traffic (web access).
Client-Server:	A model where a client (browser) requests content from a server (Nginx).
Nginx:	A fast, lightweight web server used in production.



Install and Test Nginx

```
$ sudo apt update
$ sudo apt install nginx -y
$ sudo systemctl status nginx
$ sudo systemctl enable nginx
$ echo "<h1>Welcome to Azure!</h1>" | \
  sudo tee /var/www/html/index.html
$ curl http://localhost
```

Lab 07: Deploy a Simple Website

1. SSH into your Azure Linux VM.
2. Update packages:
   `sudo apt update`
3. Install Nginx:
   `sudo apt install nginx -y`
4. Start and enable Nginx:
   `sudo systemctl enable --now nginx`
5. Create a simple web page:
   `echo "<h1>Welcome to Azure!</h1>" | sudo tee /var/www/html/index.html`
6. Access the site in your browser:
   `http://<your-vm-public-ip>`

Troubleshooting

Issue	Possible Cause and Fix
Nginx not running	• Check status: `sudo systemctl status nginx` • Start it: `sudo systemctl start nginx`
Cannot access on port 80	• Port 80 blocked by the NSG. • Allow inbound rule for TCP port 80 in the NSG.
Page not found (404)	• Wrong file path or file missing. • Ensure the file is in: `/var/www/html/index.html`

Production Connection: Understand the complete path from a browser request to a Linux web server.

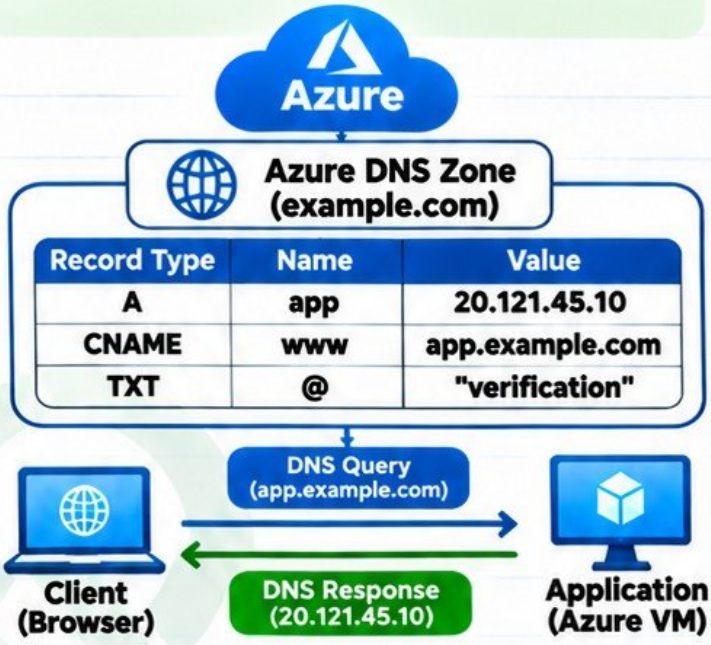


Azure DNS: Understand How Applications Get Their Names

Understand DNS records, domain names, IP addresses, and DNS resolution.

Key Concepts

DNS:	The system that translates domain names (e.g., app.example.com) to IP addresses (e.g., 20.121.45.10).
Domain Name:	A human-readable name (e.g., example.com).
DNS Record:	A mapping in DNS (e.g., A, CNAME, TXT, NS) that tells where a name points.
IP Address:	A unique address for a resource (e.g., 20.121.45.10).
DNS Resolution:	The process of finding the IP address for a domain name (e.g., using nslookup or dig).



Lab 08: Work with Azure DNS

1. Investigate a domain using nslookup:
   `nslookup example.com`
2. Investigate using dig:
   `dig example.com`
3. In the Azure Portal, go to DNS Zones → Create.
4. Create a DNS zone for your domain (e.g., yourdomain.com).
5. Add an A record:
   - Name: app
   - Type: A
   - Value: <your VM public IP>
6. Test the DNS record:
   `nslookup app.yourdomain.com`
   `dig app.yourdomain.com`

Example: DNS Lookup

```
$ nslookup app.example.com
Server:    8.8.8.8
Address:   8.8.8.8#53

Non-authoritative answer:
Name:    app.example.com
Address: 20.121.45.10
$ dig app.example.com +short
20.121.45.10
```

Troubleshooting

Issue	Possible Cause and Fix
Wrong IP returned	• Incorrect DNS record value. • Update the A record with the correct IP.
Changes not showing	• DNS caching (try: dig +trace or clear local DNS cache). • Wait for DNS propagation (a few minutes).

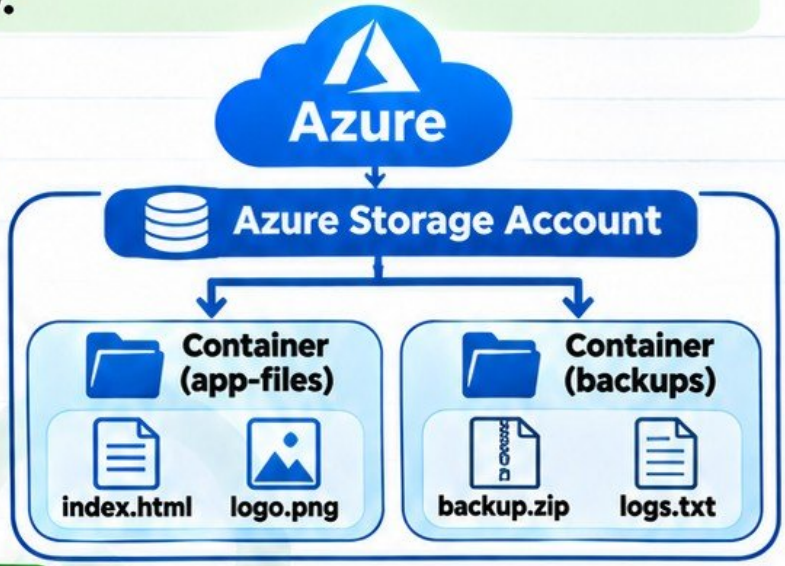
Production Lesson: A healthy application can still be unreachable when DNS is misconfigured.

Azure Blob Storage: Store and Retrieve Application Files

Understand storage accounts, containers, blobs, and storage redundancy.

Key Concepts

Storage Account:	The top-level resource that provides storage in Azure.
Container:	A logical folder inside a storage account that organizes blobs.
Blob:	A file (e.g., images, logs, backups) stored in Azure.
Storage Redundancy:	Replication of data for high availability (e.g., LRS, ZRS, GRS).



Lab 09: Work with Blob Storage

1. In the Azure Portal, go to Storage accounts → Create.
2. Set a name (e.g., veriqtastorage), choose a region, and a redundancy (e.g., LRS).
3. After creation, open the storage account and create a container (e.g., app-files) and keep it **Private.**
4. Upload a sample file (e.g., index.html) to the container.
5. Download the file using authorized access (e.g., a SAS URL or Azure CLI).

Example: Azure CLI

```
$ az storage container create \
  --name app-files --account-name \
  veriqtastorage --auth-mode login
$ az storage blob upload \
  --account-name veriqtastorage
  --container-name app-files
  --name index.html --file index.html
$ az storage blob download
  --account-name veriqtastorage
  --container-name app-files
  --name index.html --file index.html
```

Troubleshooting

Issue	Possible Cause and Fix
Access denied	• The container is private. • Use a signed URL (SAS) or proper authentication (az login).
File not found	• Incorrect container or blob name. • Verify the file path and try again.
Unexpected public access	• Public access was enabled. • Set the container access level to Private.

Security Lesson

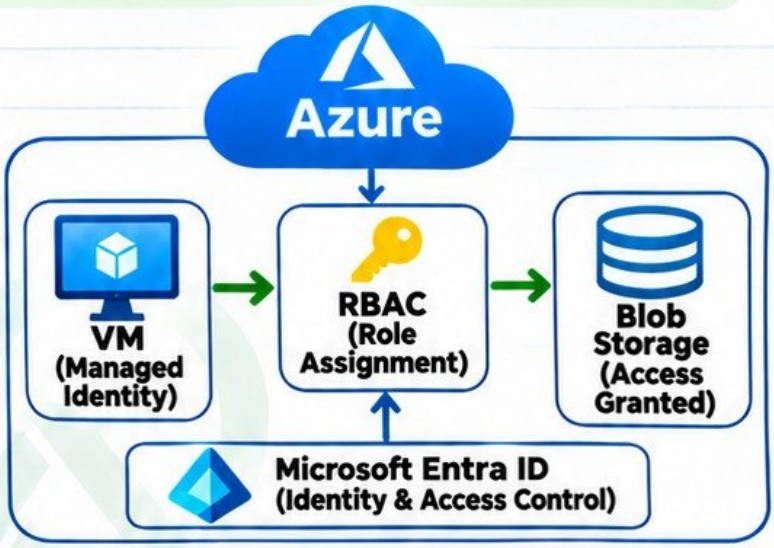
Keep storage private unless public access is explicitly required (e.g., for public files or static websites).

Managed Identities: Access Azure Without Hardcoded Passwords

- Understand Microsoft Entra ID, managed identities, and role-based access control (RBAC).
- Learn why cloud applications should avoid embedded credentials.

Key Concepts

Microsoft Entra ID:	Microsoft's identity and access management service (formerly Azure Active Directory).
Managed Identity:	A secure, Azure-managed identity for a resource (e.g., a VM) to access other Azure services.
RBAC:	Role-Based Access Control that grants the right permissions to an identity (e.g., read, write, delete).
System-Assigned:	A managed identity tied to a single Azure resource (e.g., a VM).
User-Assigned:	A reusable managed identity that can be used by multiple resources.



Example: Test Access from VM

```
$ az login --identity
$ az storage blob list \
  --account-name veriqtastorage \
  --container-name app-files \
  --output table
```

Lab 10: Configure a Managed Identity

1. In the Azure Portal, open your VM → Identity → enable **System-assigned**.
2. Go to the storage account → Access control (IAM) → **Add role assignment**.
3. Select role: **Storage Blob Data Contributor** (or a narrower role, e.g., Storage Blob Data Reader).
4. Assign the role to the VM's managed identity.
5. SSH into the VM and test access:
   `az login --identity`
   `az storage blob list --account-name <your-storage> --container-name <container>`

Troubleshooting

Issue	Possible Cause and Fix
Authorization failed (403)	• The identity exists but does not have the required role. • Assign the correct RBAC role (e.g., Storage Blob Data Contributor).
Identity not found	• Managed identity is not enabled on the VM. • Enable the system-assigned identity and retry.

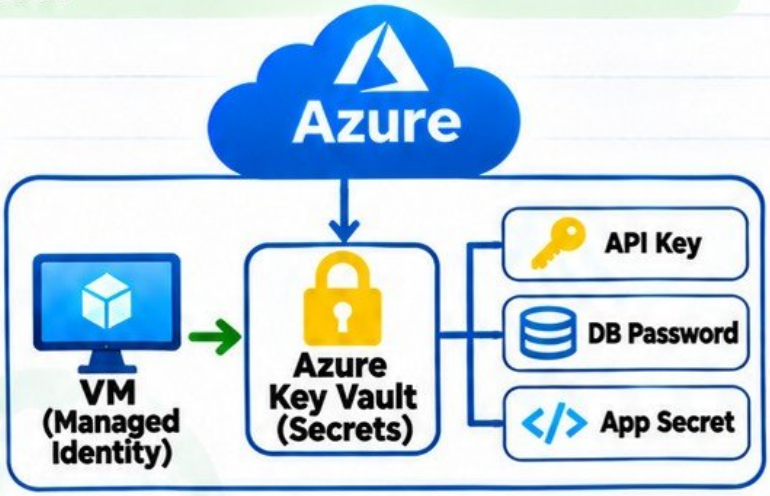
Production Lesson: Distinguish missing role assignments from authentication failures when access to Azure resources does not work.

Azure Key Vault: Protect Your First Application Secret

Understand secrets, secret versions, vault permissions, and credential exposure.

Key Concepts

Secret:	A sensitive value (e.g., API key, database password, connection string).
Secret Version:	A specific version of a secret, so you can rotate and keep history.
Key Vault:	A secure, managed Azure service for storing and controlling access to secrets, keys, and certificates.
Vault Permissions:	Access control that decides who or what can read, create, update, or delete secrets.
Credential Exposure:	When sensitive information (e.g., passwords, API keys) is unintentionally revealed (e.g., in code, logs, or repos).



Example: Store and Retrieve a Secret

```
$ az keyvault create --name veriqta-kv \
  --resource-group rg-app --location eastus
$ az keyvault secret set \
  --vault-name veriqta-kv --name db-password \
  --value "P@ssw0rd123!"
$ az keyvault secret show
  --vault-name veriqta-kv --name db-password \
  --query value -o tsv
```

Lab 11: Work with Azure Key Vault

1. In the Azure Portal, create a Key Vault (e.g., veriqta-kv).
2. Store a sample secret (e.g., db-password) with a value (e.g., P@ssw0rd123!).
3. Enable a managed identity on your VM (if not already enabled).
4. Grant the VM's managed identity access to the Key Vault (e.g., Secrets User role).
5. Retrieve the secret from the VM:
   `az keyvault secret show \`
   `--vault-name veriqta-kv \`
   `--name db-password --query value -o tsv`

Troubleshooting

Issue	Possible Cause and Fix
Forbidden (403)	• The identity does not have the required permission. • Assign the correct role (e.g., Key Vault Secrets User) or access policy.
Secret not found	• The secret name is incorrect. • Verify the secret name and that it exists in the correct Key Vault.

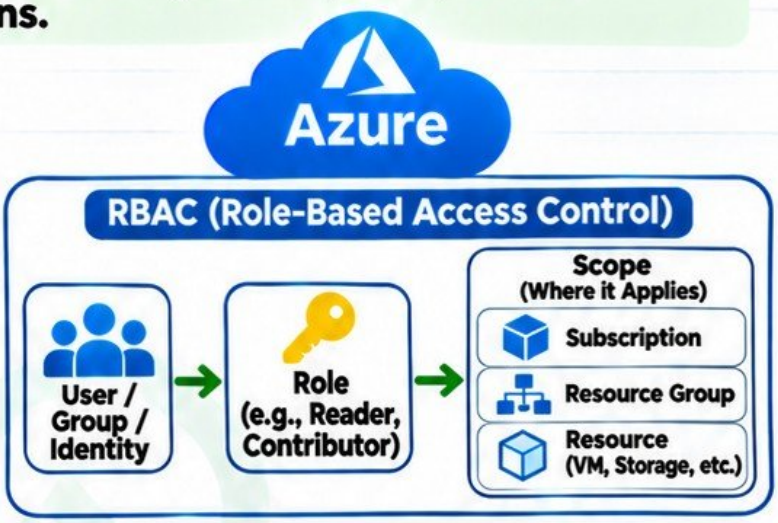
Production Lesson: Never commit secrets into Git repositories or expose them in logs.

Azure RBAC: Give Users Only the Permissions They Need

- Understand Azure roles, scopes, role assignments, and least privilege.
- Learn the difference between management-plane permissions and data-plane permissions.

Key Concepts

Role:	A collection of permissions (e.g., Reader, Contributor, Owner).
Scope:	The level where a role applies (e.g., subscription, resource group, resource).
Role Assignment:	The process of granting a role to a user, group, or managed identity at a scope.
Least Privilege:	Giving only the minimum permissions needed to do a job.
Management-Plane:	Permissions to manage Azure resources (e.g., create, update, delete resources).
Data-Plane:	Permissions to access and work with the data in a resource (e.g., read/write blobs in storage).



Common Azure Roles (Examples)

Role	What It Can Do
Reader	View Azure resources (read-only). Cannot create, update, or delete.
Contributor	Create, update, and delete most Azure resources. Cannot manage access (RBAC).
Owner	Full access, including managing access (role assignments).

Lab 12: Work with Azure RBAC

1. In the Azure Portal, go to a resource group (or subscription).
2. View current role assignments (Access control (IAM) → Role assignments).
3. Compare Reader and Contributor permissions (review the role definition).
4. Assign a limited role (e.g., Reader) to a test identity (user or group).
5. Sign in with the test identity and verify access (e.g., view a resource and try an action you are not allowed to).

Troubleshooting

Issue	Possible Cause and Fix
Authorization failed (403)	• The user/identity does not have the required role. • Assign the correct role at the right scope (e.g., resource group).
Permission not inherited	• The role was assigned at a different scope. • Verify the role assignment and cope (subscription, resource group, or resource).

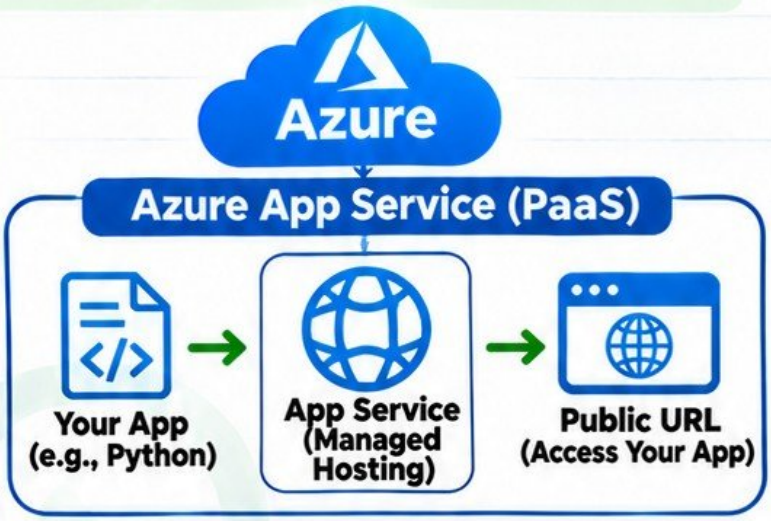
Production Connection: Restrict who can modify infrastructure, access data, and manage security settings.

Azure App Service: Deploy a Web Application Without Managing a VM

Understand platform as a service (PaaS), App Service Plans, application runtimes, and managed hosting.

Key Concepts

PaaS:	A managed cloud platform that lets you run applications without managing servers (e.g., Azure App Service).
App Service:	Azure's managed hosting service for web apps, APIs, and backend services.
App Service Plan:	The compute (CPU, memory) and pricing tier that runs your app.
Application Runtime:	The framework or language your app runs on (e.g., Python, Node.js, .NET).
Managed Hosting:	Azure handles the infrastructure, scaling, patching, and availability for you.



Common App Service Runtimes

Python | .NET | Node.js | Java | PHP

Lab 13: Deploy a Python Web App

1. Create an App Service Plan (e.g., Basic or Standard).
2. Create an App Service Web App (e.g., veriqta-app) and select Python (e.g., Python 3.12) as the runtime.
3. Prepare a simple Python web app (e.g., Flask):

```
app/
├── app.py
└── requirements.txt
```

4. Deploy the app (zip deploy or Git) to the App Service.
5. Test the public endpoint: https://veriqta-app.azurewebsites.net

Troubleshooting

Issue	Possible Cause and Fix
App fails to start (502/503)	• Incorrect application runtime (e.g., wrong Python version). • Missing dependencies (e.g., requirements.txt not installed). • Check the App Service logs (Log stream).
Runtime error	• Application error in the code (e.g., startup failure). • Review logs and fix the error.
App not reachable	• The app is not running or failed to deploy. • Verify the App Service is Running and the URL is correct.

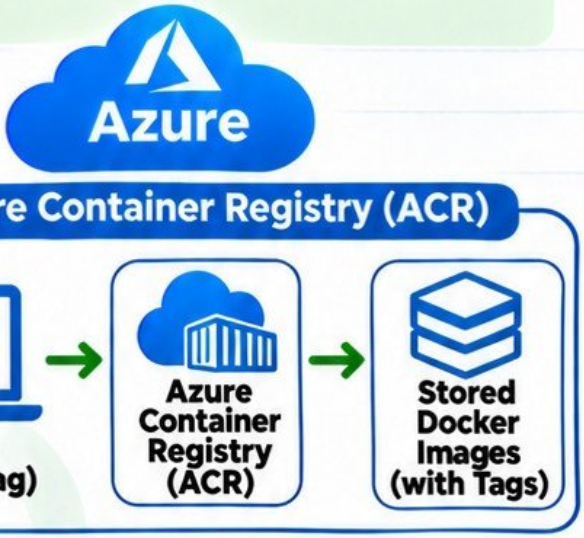
Production Lesson: Use Azure App Service when you want faster deployment, automatic scaling, and less operational work than a VM.

Azure Container Registry: Store Your First Docker Image

Understand Docker images, containers, registries, image tags, and authentication.

Key Concepts

Docker Image:	A read-only blueprint of your application (e.g., app:v1).
Container:	A running instance of an image.
Registry:	A centralized service that stores and manages Docker images (e.g., Azure Container Registry).
Image Tag:	A version label for an image (e.g., v1, v2, latest).
Authentication:	The process of securely signing in to a registry to push or pull images (e.g., Azure login).



Common Docker Image Tags

Tag	Meaning
latest	The most recent version (use with care).
v1	A specific, fixed version.
v1.0.0	A version with a clear release number.

Lab 14: Push an Image to ACR

1. Create an Azure Container Registry (e.g., veriqtaacr) in the Azure Portal.
2. Prepare a simple app and Dockerfile (e.g., a Python Flask app).

```
app/
├── app.py
└── Dockerfile
```

3. Build the Docker image:

```
$ docker build -t veriqta-app:v1 .
```

4. Sign in to ACR:

```
$ az acr login --name veriqtaacr
```

5. Tag the image for ACR:

```
$ docker tag veriqta-app:v1 \
  veriqtaacr.azurecr.io/veriqta-app:v1
```

6. Push the image:

```
$ docker push \
  veriqtaacr.azurecr.io/veriqta-app:v1
```

7. Verify in the Azure Portal: ACR → Repositories → veriqta-app → Tags.

Troubleshooting

Issue	Possible Cause and Fix
Authentication failed (unauthorized)	• You are not signed in to ACR. • Run: az acr login --name <registry-name>.
Image not found	• The image name or tag is incorrect. • Check the exact repository name and tag, then push again.
Push permission denied	• Your identity lacks the required role (e.g., AcrPush). • Assign the AcrPush or Contributor role on the registry and retry.

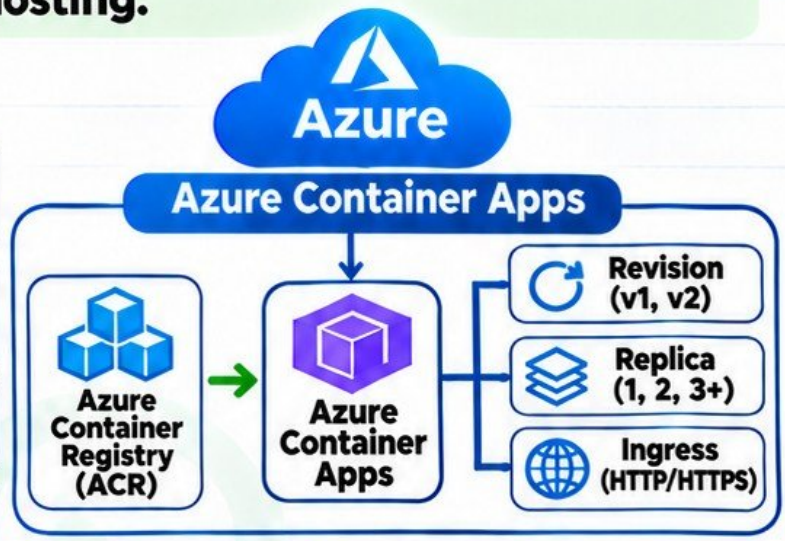
Production Connection: Container registries supply deployable application artifacts for DevOps pipelines and deployment.

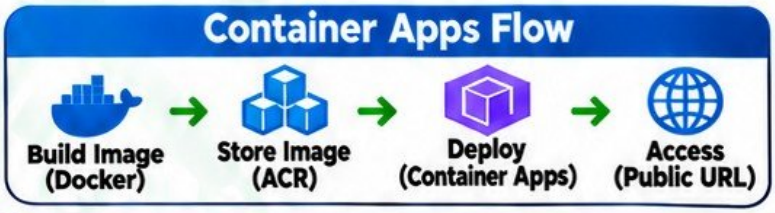
Azure Container Apps: Run Your First Containerized Application

Understand container revisions, ingress, replicas, and managed container hosting.

Key Concepts

Container App:	A fully managed Azure service to run containerized applications without managing servers.
Revision:	A version of your app (e.g., v1, v2) created with a new configuration or image.
Ingress:	The public access point that exposes your app (HTTP/HTTPS).
Replica:	The number of running instances of your app (e.g., 1, 2, 3) for scale and availability.
Managed Hosting:	Azure handles the infrastructure, scaling, networking, and updates for you.





Lab 15: Deploy to Container Apps

1. Use an existing Docker image in Azure Container Registry (e.g., veriqta-app:v1).
2. Create an Azure Container App (e.g., veriqta-containerapp) and select the image from ACR.
3. Configure ingress:
   - Enable external access (HTTP/HTTPS)
   - Set the application port (e.g., 8080)
   - Set replicas (e.g., 1 or 2)
4. Deploy the app and wait for it to provision.
5. Verify the endpoint (e.g., https://veriqta-containerapp.azurecontainerapps.io) and confirm the application is running.

Troubleshooting

Issue	Possible Cause and Fix
Image pull failed	• Incorrect image name or tag. • Missing ACR permissions (e.g., AcrPull role). • Verify the image exists in ACR and the app has access.
Unhealthy revision	• The application failed to start (e.g., incorrect command or port). • Check container logs and fix the startup error.
Application not accessible	• Ingress not enabled or wrong port. • Verify ingress is set to external and the correct port is configured.

Production Lesson: Deploying an image is not enough. Engineers must verify that the application actually works.

Infrastructure as Code: Deploy Azure Resources Using Bicep

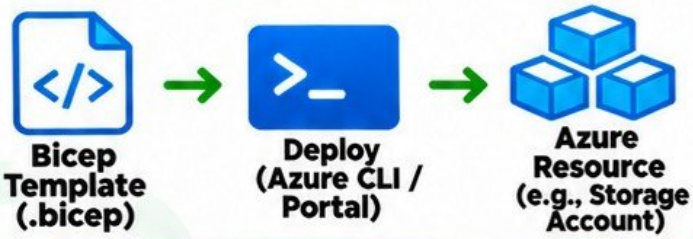
Understand infrastructure as code, declarative configuration, parameters, and repeatable deployments.

Key Concepts

Infrastructure as Code (IaC):	Manage and deploy infrastructure using code (e.g., Bicep, ARM, Terraform).
Bicep:	A simple, declarative language for deploying Azure resources.
Declarative Configuration:	You describe the desired state, and Azure creates or updates it.
Parameters:	Input values (e.g., name, location, SKU) to make templates reusable.
Repeatable Deployments:	Deploy the same infrastructure consistently and reliably (dev, test, prod).



Azure Deployment with Bicep



Example: Simple Bicep (Storage Account)

```
param storageName string
param location string = resourceGroup().location

resource storage 'Microsoft.Storage/storageAccounts@2023-01-01' = {
  name: storageName
  location: location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
}
```

Lab 16: Deploy with Bicep

1. Create a Bicep file (main.bicep) for a storage account.
2. Define parameters (e.g., storage name and location).
3. Deploy the template:

```
az deployment group create \
--resource-group veriqta-rg \
--template-file main.bicep
```

4. Check the deployment result in the Azure Portal.
5. Update a configuration (e.g., change SKU) and redeploy to apply the change safely.

Troubleshooting

Issue	Possible Cause and Fix
Template validation error	• Syntax or property error in the Bicep file. • Run: az bicep build main.bicep to validate and fix errors.
Deployment failed	• Invalid parameter value, name conflict, or unsupported SKU. • Check the deployment error details and correct the configuration.
Resource not updated	• The change was not included in the template. • Update the template and redeploy (using a new or updated parameter).

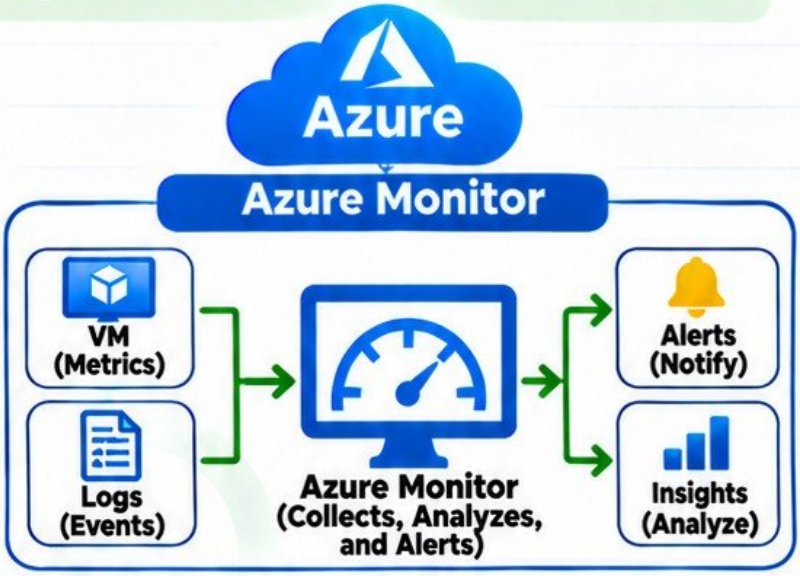
Production Connection: Version-controlled infrastructure reduces configuration drift and manual mistakes.

Azure Monitor: Detect Problems Before Users Report Them

Understand metrics, logs, alerts, and monitoring signals.

Key Concepts

Metrics:	Numeric data that shows resource health and performance (e.g., CPU, memory, disk).
Logs:	Detailed, time-stamped events that help with investigation and troubleshooting (e.g., VM logs, activity logs).
Alerts:	Rules that watch metrics or logs and notify you when conditions are met (e.g., high CPU usage).
Monitoring Signal:	A specific type of data (metric, log, or event) used for monitoring and alerting.
Azure Monitor:	A unified service in Azure to collect, analyze, and act on metrics, logs, and alerts across your resources.



Monitoring Signals in Azure Monitor

Metrics	Logs	Alerts
(e.g., CPU, Memory)	(e.g., Activity, VM)	(e.g., High CPU)

Lab 17: Monitor a VM and Create an Alert

1. Open the Azure Portal and go to your virtual machine (VM).
2. View VM metrics (e.g., CPU percentage, memory usage).
3. Create a metric alert:
   - Scope: your VM
   - Metric: CPU Percentage
   - Condition: Greater than 80%
   - Evaluation: Every 5 minutes
   - Action: Email (or notification)
4. Save the alert and verify it is created.
5. Test the setup (e.g., generate higher CPU load) and confirm the alert is triggered.

Troubleshooting

Issue	Possible Cause and Fix
Missing metrics	• The VM or resource is not sending metrics. • Wait a few minutes, check the resource status, and ensure monitoring is enabled.
Alert not triggering	• Threshold is too high or evaluation frequency is too long. • Lower the threshold (e.g., 80%) and confirm the evaluation settings.
Unexpected alerts (false alarms)	• Normal spikes in usage can trigger alerts. • Adjust the threshold, add a longer evaluation period, or use a more specific metric.

Production Lesson: Monitoring should provide actionable evidence, not just attractive dashboards.

Azure Logs: Investigate a Real Application Failure

Understand application logs, system logs, Log Analytics workspaces, and Kusto Query Language (KQL).

Key Concepts

Application Logs:	Logs from your app (e.g., errors, warnings, requests, responses).
System Logs:	Logs from the underlying system (e.g., OS, services, VM events).
Log Analytics Workspace:	A centralized place in Azure to collect, store, and query logs at scale.
KQL (Kusto Query Language):	A powerful query language used to search, filter, analyze, and correlate log data.

Azure Logs and Investigation Flow



Example: Basic KQL Query

```
AppRequests
| where ResponseCode >= 500
| project TimeGenerated, OperationName,
  ResponseCode, ResultDescription
| order by TimeGenerated desc
```

Lab 18: Investigate a Failure with Logs

1. Ensure logs are being sent to a Log Analytics workspace (e.g., from a VM or App Service).
2. Generate a controlled failure (e.g., access a non-existent route or stop a required service).
3. Open Log Analytics and run a KQL query to find the error:

```
AppRequests
| where ResponseCode >= 500
| project TimeGenerated, ResultDescription
| order by TimeGenerated desc
```

4. Review the results and identify the root cause (e.g., application error, missing dependency).
5. Document the findings and the corrective action you would take.

Troubleshooting

Issue	Possible Cause and Fix
No logs found	• Log collection is not enabled (e.g., VM agent or diagnostics). • Enable and verify the correct workspace and data source.
Incomplete logs	• Incorrect diagnostic settings or wrong log category selected. • Update the diagnostic configuration and allow time for data ingestion.
Incorrect results in KQL	• Wrong table, filter, or time range. • Check the correct table (e.g., AppRequests, AzureActivity) and adjust the query and time window.

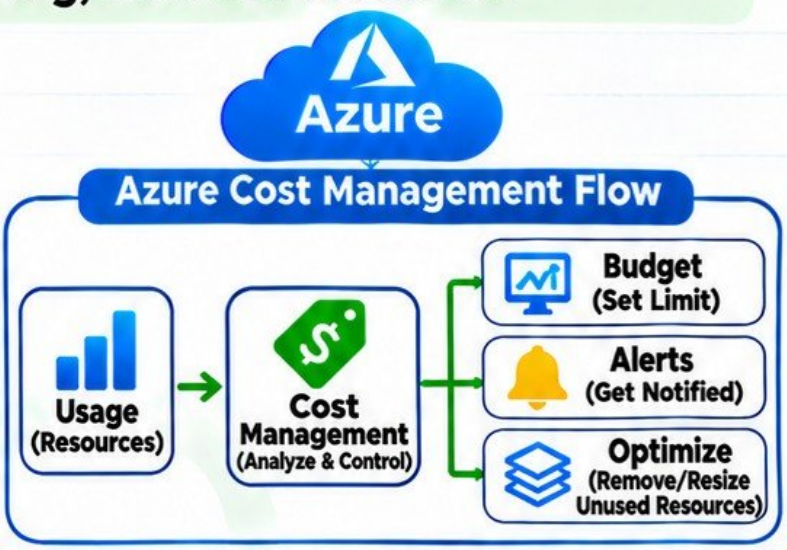
Production Lesson: Engineers should identify the cause using evidence before changing infrastructure.

Azure Cost Management: Find Waste and Prevent Unexpected Bills

Understand cloud billing, consumption-based pricing, budgets, resource sizing, and cost allocation.

Key Concepts

Cloud Billing:	The cost of using Azure resources, based on actual consumption (e.g., compute, storage, data).
Consumption-Based Pricing:	You pay for what you use (e.g., per hour, per GB, per request).
Budget:	A spending limit you set with alerts to notify you when usage reaches a threshold.
Resource Sizing:	Choosing the right size (e.g., VM, storage, database) for your workload and cost.
Cost Allocation:	Assigning and tracking costs by resource, project, team, or tag (e.g., environment, app, owner).



Example: Budget Alert Configuration

Budget: $100	Alert: 80% (=$80)	Notify: Email	Action: Review and Optimize

Lab 19: Review Costs and Set a Budget

1. Open Cost Management + Billing in the Azure Portal.
2. Review subscription costs (e.g., by service, resource group, or tag).
3. Create a budget and set an alert (e.g., $100 budget, 80% alert).
4. Identify unused or idle resources (e.g., stopped VMs, unneeded disks, old test resources).
5. Review estimated ongoing charges and take action to reduce waste (e.g., delete, resize, or stop resources).

Troubleshooting

Issue	Possible Cause and Fix
Higher than expected cost	• Always-on or idle resources (e.g., running VMs, unused disks). • Identify and stop, resize, or delete unused resources.
Budget alert not received	• Incorrect alert threshold or notification setup. • Verify the budget, threshold (e.g., 80%), and email recipients.
Unexplained charges	• Forgotten resources, data transfer, or higher resource tier. • Review detailed cost breakdown and resource usage history.

Production Lesson: Cost management is part of responsible cloud engineering. Budget alerts do not automatically stop resources.

Final Azure Challenge: Deploy, Break, Diagnose, and Recover

Scenario: A small company needs a publicly accessible web application hosted on Azure. The application must be secured, monitored, and recoverable from a basic configuration failure.

Key Concepts

Deployment:	Host a web application using Azure App Service or Azure Container Apps.
Security:	Apply the right access permissions and network controls.
Monitoring:	Use Azure Monitor and logs to observe application health and issues.
Failure Diagnosis:	Use logs and evidence to identify the root cause.
Recovery:	Restore the correct configuration and verify the application.

Azure

Solution Overview (High Level)



Troubleshooting

Issue	Possible Cause and Fix
Application not loading	• Incorrect configuration (e.g., port, startup command, or app setting). • Check app settings and logs, then update and restart.
Access denied (403/401)	• Missing or incorrect access permissions (e.g., RBAC or networking rules). • Verify role assignments and network access (e.g., NSG or access restrictions).
No metrics or logs	• Monitoring or log collection not enabled. • Enable Azure Monitor / Log Analytics and validate data ingestion.

Lab 20: Guided Cloud Operations Challenge

1. Deploy a web application using Azure App Service or Azure Container Apps.
2. Configure the necessary networking and access permissions.
3. Verify the application endpoint (e.g., https://<your-app>.azurewebsites.net).
4. Enable appropriate monitoring (e.g., Azure Monitor) and inspect logs (e.g., Log Analytics).
5. Intentionally introduce a safe configuration error (e.g., wrong app setting or port).
6. Observe the failure, investigate the evidence (metrics and logs), and identify the root cause.
7. Restore the working configuration and verify recovery.
8. Remove unnecessary resources (e.g., test resources, idle plans).

Final Deliverable

A GitHub portfolio project containing:

- A simple architecture diagram.
- Deployment steps (step-by-step).
- Sanitized screenshots (deployment, monitoring, failure, and recovery).
- Troubleshooting evidence (logs and metrics).
- A short incident report (what happened, root cause, fix, and lessons learned).

Production Lesson: Engineers must be able to deploy, diagnose, recover, and clean up resources in a real-world scenario.